

TuneHUD

Engineering Design Document

Version 0.1 | Draft

Control system tuning platform — architecture, protocol spec, and plugin interface

1. Overview

TuneHUD is a cross-platform control system tuning and instrumentation platform. It connects to industrial hardware over standard protocols and provides live visualization, parameter tuning, session logging, and video/vision capabilities through iOS, Android, and web clients.

The system is designed around three principles:

- Transport agnostic — any hardware that speaks Modbus, HART, BLE, WebSocket, or serial is a first-class citizen.
- Rate-limited observation — TuneHUD observes running control loops at 10–100 Hz. It does not close control loops. The controller under test runs at its own rate on its own hardware.
- Queryable params — every hardware connection exposes a manifest of readable and writable parameters. The app builds its UI dynamically from the manifest. No hardcoded UI per device.

2. System Architecture

2.1 Layers

Layer	Description
UI	iOS (SwiftUI), Android (Jetpack Compose), Web (browser). Connects to backend over WebSocket. Renders UI from p
Gateway	Python/asyncio process on Raspberry Pi 5 (or any Linux host). WebSocket server, param router, data logger, rate limit
Transport	Pluggable drivers: Modbus TCP/RTU, HART, BLE, NI DAQmx, serial/UART. Each implements the plugin interface.
Hardware	The actual device under test: PLC, field instrument, MCU, NI DAQ hardware, etc.

2.2 Gateway host

The gateway runs on a Raspberry Pi 5. No real-time OS is required. The system is rate-limited to 10–100 Hz at the WebSocket layer, well within what asyncio + Python can sustain. The Hailo-8 accelerator on the Pi 5 handles vision inference for gauge OCR and video processing.

For portable or battery-powered deployments, a Pi Zero 2W is sufficient for non-vision use cases.

2.3 WebSocket protocol

All clients connect over WebSocket. The protocol is JSON-framed with the following top-level message types:

Message type	Description
manifest_req	Client requests the param manifest from a named transport.
manifest_resp	Gateway returns the manifest for the requested transport.
stream_start	Client subscribes to a list of param names at a given Hz.

stream_data	Gateway pushes timestamped param values to the client.
param_write	Client sends a write request for a named param.
write_ack	Gateway confirms the write completed (or returns an error).
error	Gateway reports a transport or protocol error.

3. Param Manifest Specification

3.1 Purpose

The param manifest is the contract between a transport plugin and the UI. When a client connects, it sends a manifest_req for a transport. The gateway queries the plugin and returns a manifest_resp containing all available parameters. The UI builds its tuning interface entirely from this manifest — no hardcoded widgets per device.

3.2 Manifest structure

```
{
  "transport": "modbus_tcp",
  "device":    "Acme PLC model 3000",
  "version":   "1.0",
  "params":    [ , ... ]
}
```

3.3 ParamDescriptor fields

Field	Type	Description
name	string	Unique identifier. Used as key for read/write/stream. e.g. "kp", "setpoint"
label	string	Human-readable display name. e.g. "Proportional gain"
type	string	Data type: "float", "int", "bool", "enum"
units	string null	Engineering units. e.g. "degC", "rpm", "%", null
min	number null	Minimum valid value. Null if unconstrained.
max	number null	Maximum valid value. Null if unconstrained.
step	number null	Suggested increment for jog/nudge UI.
read_only	bool	True if parameter can be read but not written.
group	string null	Optional grouping label for UI. e.g. "PID gains"
enum_values	array null	Required if type is "enum". Array of {value, label}.
description	string null	Optional longer description for tooltip/help.

3.4 Example manifest

```
{
  "transport": "serial",
  "device": "MyController v2",
  "version": "1.0",
  "params": [
    { "name": "kp", "label": "Proportional gain", "type": "float",
      "units": null, "min": 0.0, "max": 100.0, "step": 0.01,
      "read_only": false, "group": "PID gains" },
    { "name": "setpoint", "label": "Setpoint", "type": "float",
      "units": "degC", "min": 0.0, "max": 200.0, "step": 0.5,
      "read_only": false, "group": "Control" },
    { "name": "pv", "label": "Process value", "type": "float",
      "units": "degC", "min": null, "max": null, "step": null,
      "read_only": true, "group": "Control" }
  ]
}
```

4. Transport Plugin Interface

4.1 Required methods

```
class TransportPlugin:
    async def connect(self) -> None
    async def disconnect(self) -> None
    async def get_manifest(self) -> dict
    async def read_param(self, name: str) -> Any
    async def write_param(self, name: str, value: Any) -> None
    async def read_all(self) -> dict[str, Any]
```

4.2 Register map (Modbus and HART)

For Modbus and HART plugins, the user supplies a YAML register map that binds param names to register addresses. The register map decouples hardware addressing from the param namespace — clients never see register numbers.

```
# register_map.yaml
params:
  kp:
    register: 100
    type: float32
    scale: 0.001
  setpoint:
    register: 102
    type: float32
    scale: 0.1
  pv:
    register: 200
    type: float32
    scale: 0.1
    read_only: true
```

5. Data Flow

5.1 Streaming

When a client sends `stream_start` with a list of param names and a target Hz, the gateway starts a polling loop at the requested rate (capped at the system maximum). Data is pushed as `stream_data` messages:

```
{
  "type": "stream_data",
  "t":    1717100000.123,
  "data": { "kp":1.25, "setpoint":75.0, "pv":74.3 }
}
```

5.2 Param writes

The client sends a `param_write`. The gateway validates against the manifest (type, min, max, read_only) before forwarding to the plugin. On success, a `write_ack` returns the actual value read back from hardware. Every param write is appended to the session log with timestamp, previous value, and new value.

```
// Client sends:
{ "type":"param_write", "name":"kp", "value":1.5 }

// Gateway responds:
{ "type":"write_ack", "name":"kp", "value":1.5, "t":1717100001.456 }
```

6. Session Logging

The data logger writes two files per session:

- `session_YYYYMMDD_HHMMSS.csv` — timestamped stream data, one row per `stream_data` message.
- `session_YYYYMMDD_HHMMSS_writes.csv` — one row per param write: timestamp, param name, previous value, new value, client ID.

Both files are written incrementally so data is not lost if the session terminates unexpectedly.

7. Rate Limiting

The system does not need a real-time OS because it does not close control loops. The gateway imposes a configurable maximum stream rate (default 50 Hz, max 100 Hz):

```
# tunehud.yaml
stream:
  max_hz:    100
  default_hz: 50
```

8. Supported Transports

Transport	Status	Notes
Modbus TCP	Planned v1.0	pymodbus. Register map via YAML.
Modbus RTU	Planned v1.0	pymodbus over serial or USB-RS485.
HART	Planned v1.1	Requires HART modem (AD5700). Bell 202 FSK over 4-20 mA loop.
BLE	Planned v1.0	bleak library. GATT characteristic map via YAML.
WebSocket	Planned v1.0	For devices that already speak TuneHUD protocol natively.
Serial/UART	Planned v1.0	Configurable frame format via YAML.
NI DAQmx	Planned v1.2	nidaqmx Python package. Windows host required for NI driver.
CAN bus	Planned v2.0	python-can. DBC file for signal mapping.

9. Vision Features

9.1 Live video

The gateway streams MJPEG from a USB camera or Pi camera module. Forwarded to connected clients as a separate WebSocket binary stream. Clients render video alongside plots.

9.2 Gauge OCR

The vision pipeline accepts a camera frame and a calibration file describing a gauge or panel meter. It returns a numeric reading injected into the stream as a virtual param, indistinguishable from hardware params.

- 7-segment and LCD displays — Apple Vision framework on iOS, or Tesseract on Pi.
- Analog needle gauges — angle detection via OpenCV. Requires one-time calibration: user marks zero and full-scale positions in the app.

The Hailo-8 accelerator on the Pi 5 is used for inference when available.

10. Reference Hardware

Role	Hardware
Gateway (primary)	Raspberry Pi 5, 4 GB RAM, with Hailo-8 M.2 accelerator. Ubuntu 24.04.
Gateway (portable)	Raspberry Pi Zero 2W. Non-vision use cases only.
HART modem	AD5700 or MAX14459 FSK modem. Connects to Pi via SPI.
RS-485 adapter	USB-RS485 dongle for Modbus RTU.
TuneHUD DAQ (optional)	ESP32 + ADS131M08 16-bit ADC. Speaks TuneHUD WebSocket protocol natively. 8 analog inputs

11. Open Questions

- Android client: native Jetpack Compose or Flutter shared codebase with iOS?
- NI DAQmx: Windows-only driver. Pi as gateway not an option for NI hardware — needs Windows sidecar or direct laptop connection.
- HART modem hardware: validate AD5700 SPI timing on Pi 5 before committing to that part.
- Gauge OCR calibration UX: needs a first-time setup flow in the app.
- Session file location and rotation policy on the Pi.